

BhuNexis - Full-Stack Architecture Documentation

Government Land Records Digitization Platform & Interactive PostGIS Cadastral GIS Solution

SIH 2026 Hackathon

Python FastAPI

React 18 + Leaflet

PostgreSQL + PostGIS

Enterprise Solution

1. Executive Summary & Core Objective

BhuNexis is an enterprise-grade government land records digitization platform designed to solve legacy land revenue record errors, enable AI-driven OCR document extraction, provide human-in-the-loop validation, and render high-resolution **PostGIS Cadastral GIS Maps** linked directly to survey numbers, khasra numbers, khata numbers, and landowner identities.

This technical specification document outlines the complete codebase architecture, database schema, API contracts, frontend component tree, and PostGIS GIS Cadastral mapping subsystem so that developers and AI systems can seamlessly understand, execute, and extend the project.

2. Technology Stack & Frameworks

Domain	Technology / Library	Version	Purpose & Usage
Backend Runtime	Python	3.13	Core execution runtime environment for backend APIs and processing logic.
Web Framework	FastAPI	0.141	High-performance async web framework providing RESTful API endpoints & OpenAPI docs.
Database ORM	SQLAlchemy	2.0.54	Object-Relational Mapping with typed Mapped columns, relationships, and queries.
Database Engine	PostgreSQL + PostGIS	15+	Relational spatial database storing structured land records & vector polygon geometries.
Database Drivers	psycopg 3 / GeoAlchemy2	3.3.6	PostgreSQL dialect driver and PostGIS spatial geometry mapping engine.
Authentication	PyJWT & bcrypt	2.14 / 5.0	JWT Bearer token creation, validation, password hashing, and Role-Based Access Control.
Object Storage	MinIO Python SDK	7.2.20	S3-compatible object storage client for raw scanned PDF land record documents.
Frontend Engine	React 18 + Vite	18.x	Modern component-based UI runtime with fast module replacement build system.
GIS Mapping	Leaflet & React-Leaflet	1.9.x	Interactive map rendering engine for PostGIS GeoJSON land parcel polygon boundaries.
Styling & Icons	TailwindCSS + Lucide React	3.x	Utility-first CSS styling system, responsive grid layouts, custom dark/light themes.
HTTP Client	Axios	1.x	Client API service layer with JWT request interceptor & error handling.

3. Complete Folder & File Directory Structure

```

BhuNexis/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── deps.py          # Dependency injection (Auth, DB, RBAC role guard)
│   │   │   └── v1/
│   │   │       ├── api.py        # Main API router aggregator
│   │   │       └── endpoints/
│   │   │           ├── auth.py    # POST /auth/login, POST /auth/register, GET /auth/me
│   │   │           ├── citizen.py # Citizen parcel search & verification API
│   │   │           └── dashboard.py # Role dashboards (Citizen, Reviewer, Officer, Admin, Auditor)
│   │   ├── documents.py          # Document upload, storage, OCR triggering
│   │   ├── integrations.py       # External OCR/NLP API webhook receivers
│   │   ├── map.py                # GET /map/parcels PostGIS GeoJSON spatial endpoint
│   │   ├── parcels.py            # Land parcel CRUD & spatial relationship API
│   │   └── reviews.py            # Human-in-the-loop review queue & field corrections
│   ├── core/
│   │   ├── audit.py              # System audit logger for compliance events
│   │   ├── config.py             # Pydantic environment configuration & settings
│   │   ├── rbac.py               # AppRole enum & role permission definitions
│   │   ├── security.py           # Password hashing (bcrypt) & JWT token handling
│   │   └── storage.py             # MinIO / Local storage file manager
│   ├── db/
│   │   ├── base.py               # SQLAlchemy DeclarativeBase initialization
│   │   └── session.py            # Database engine & session maker
│   ├── models/
│   │   └── all_models.py          # Unified SQLAlchemy DB models (User, Owner, Parcel, etc.)
│   ├── schemas/
│   │   ├── auth.py               # Auth request & token response schemas
│   │   ├── document.py           # Document upload & OCR page schemas
│   │   ├── parcel.py             # Parcel, GeoJSON feature & feature collection schemas
│   │   └── review.py             # Review case & action schemas
│   └── main.py                   # FastAPI application entrypoint & CORS middleware
├── tests/                       # Pytest suite (test_api.py, test_health.py)
├── pyproject.toml                # Python package configuration
├── pytest.ini                   # Test runner configuration
└── frontend/
    ├── src/
    │   ├── components/
    │   │   ├── common/           # Reusable UI widgets (StatusBadge, Navbar, Footer)
    │   │   ├── layout/           # Sidebar layout & Header controls
    │   │   └── map/
    │   │       └── MapViewer.jsx  # Cadastral GIS Map viewer with parcel lines & plot labels
    │   ├── pages/
    │   │   ├── Home.jsx          # Landing page & platform overview
    │   │   ├── Login.jsx         # Role-based JWT authentication page
    │   │   ├── MapViewPage.jsx   # Fullscreen Cadastral GIS Map page
    │   │   ├── admin/            # User Management & System Settings
    │   │   ├── auditor/          # Audit Logs & Security Analytics
    │   │   ├── citizen/          # Citizen Search & Land Rights
    │   │   ├── dashboards/       # Citizen, Reviewer, Officer, Admin, Auditor Dashboards
    │   │   ├── officer/          # Upload Land Record & Validation Workflows
    │   │   └── reviewer/         # Human-in-the-loop OCR verification page
    │   └── routes/
    │       ├── AppRoutes.jsx     # React Router v6 route configuration
    │       └── ProtectedRoute.jsx # Role-based route guard wrapper

```

```
├── services/
│   ├── apiClient.js      # Axios client with JWT bearer interceptors
│   ├── authService.js    # Login, register, auth helper functions
│   └── mapService.js      # API service for PostGIS GeoJSON endpoints
├── App.jsx               # Application root component
├── main.jsx              # Vite React entry point
├── package.json          # Frontend npm dependencies
└── vite.config.js        # Vite server & API proxy config
```

4. Core Database Schema & PostGIS Architecture

The system utilizes PostgreSQL with two schemas: `core` for relational entities and `gis` for PostGIS spatial geometries.

Table Name	Schema	Key Columns & Description
<code>core.users</code>	core	<code>id</code> , <code>email</code> , <code>password_hash</code> , <code>full_name</code> , <code>role</code> (CITIZEN, REVIEWER, OFFICER, ADMIN, AUDITOR), <code>is_active</code>
<code>core.owners</code>	core	<code>id</code> , <code>owner_uid</code> , <code>full_name</code> , <code>local_name</code> , <code>father_name</code> , <code>gender</code> , <code>address</code> , <code>village</code> , <code>tehsil</code> , <code>district</code> , <code>state</code>
<code>core.parcels</code>	core	<code>id</code> , <code>parcel_uid</code> , <code>survey_number</code> , <code>khasra_number</code> , <code>khata_number</code> , <code>plot_number</code> , <code>village</code> , <code>tehsil</code> , <code>district</code> , <code>area</code> , <code>status</code>
<code>core.land_rights</code>	core	<code>id</code> , <code>owner_id</code> (FK), <code>parcel_id</code> (FK), <code>ownership_type</code> , <code>share_percentage</code> , <code>is_current</code>
<code>gis.parcel_geometry</code>	gis	<code>id</code> , <code>parcel_id</code> (FK), <code>geometry</code> (Geometry(Polygon,4326)), <code>geometry_source</code> , <code>gis_area_sq_m</code>
<code>core.documents</code>	core	<code>id</code> , <code>document_uid</code> , <code>title</code> , <code>file_path</code> , <code>file_type</code> , <code>ocr_status</code> , <code>uploaded_by</code> (FK)
<code>core.extracted_fields</code>	core	<code>id</code> , <code>document_page_id</code> (FK), <code>field_name</code> , <code>extracted_value</code> , <code>confidence_score</code> , <code>validation_status</code>
<code>core.review_cases</code>	core	<code>id</code> , <code>parcel_id</code> (FK), <code>document_id</code> (FK), <code>assigned_to</code> (FK), <code>priority</code> , <code>status</code> , <code>reviewer_comment</code>

5. GIS & Cadastral Mapping Subsystem Implementation

The Cadastral Map subsystem enables seamless spatial visualization of land parcels stored in PostGIS:

- **Backend GeoJSON Endpoint** (`backend/app/api/v1/endpoints/map.py`): Executes PostGIS query `ST_AsGeoJSON(pg.geometry)::json` joined with `core.parcels` to extract vector boundary polygons and attaches plot numbers, khasra numbers, survey numbers, land use classification, and verification status to GeoJSON feature properties.
- **Frontend Map Viewer** (`frontend/src/components/map/MapView.jsx`): Uses `React-Leaflet` to render parcel polygons. Includes:
 - **Cadastral Line Demarcation**: Custom stroke styling (Emerald line for Verified, Crimson line for Review Required, Amber dashed line for Draft).
 - **Permanent Plot Labels**: Displays permanent plot number badges (e.g. `PLT-04011`) centered on each land parcel polygon using Leaflet tooltips.
 - **Auto-Fit View Bounds**: Built-in `AutoFitBounds` component calls `map.fitBounds()` to automatically center and zoom to fit loaded PostGIS parcels.
 - **Base Layer Switcher**: Toggle between Standard (OSM), Satellite Imagery (Esri World Imagery), and Dark GIS canvas.
 - **Interactive Detail Modal**: Clicking any parcel highlights its border and slides up a detailed card showing plot, khasra, khata numbers, area, and owner info.

6. Recent Fixes & Improvements

- **Backend Model Properties & Setter Fixes**: Added missing `@reviewer_notes.setter` on `ReviewCase` and `@status.setter` on `ExtractedField` in `all_models.py` to prevent read-only property assignment errors.
- **SQLAlchemy Query Filter Correction**: Updated `reviews.py` to join `DocumentPage` explicitly, resolving filter parameter type mismatches.
- **Database Schema Migration**: Executed DDL migration `ALTER TABLE core.owners ADD COLUMN IF NOT EXISTS owner_uid VARCHAR;` and backfilled UIDs for 60 existing owner records, resolving HTTP 500 exceptions on citizen dashboard APIs.
- **Comprehensive Unit Testing**: Verified all API endpoints using `pytest`, achieving 100% test pass rate.